

Tutorial: Deep Learning for Happy vs. Not Happy Facial Expression Recognition in Video

Jose Carlos Alania Aguero
Dept. Electrical & Computing
Engineering
Image Processing ECE-533 2026
Albuquerque, NM USA
jalaniaguero99@unm.edu

Abstract—Facial emotion recognition in video is a rapidly growing field with applications in human-computer interaction, mental health monitoring, and multimedia analysis. This tutorial presents a step-by-step framework for classifying speaking faces into two categories: *happy* and *not happy* (including serious, neutral, and crying expressions). We focus on the use of ResNet as a backbone for feature extraction, leveraging its proven ability to capture subtle facial cues in video frames [0]. The tutorial guides readers through dataset selection, preprocessing (face detection, alignment, normalization, and class balancing), and model training on video-based emotion datasets [1]. Validation strategies using complementary datasets are discussed to ensure generalization, while testing approaches highlight robustness in real-world conditions [2]. The tutorial emphasizes practical implementation on accessible platforms such as Google Colab, making it suitable for students and researchers with limited computational resources. Finally, we outline future directions, including the integration of temporal modeling with ResNet + LSTM, to capture dynamic emotional transitions across video sequences [3]. This work provides a comprehensive and accessible pathway for building emotion classification systems, bridging theory and practice in deep video analysis. *act*.

Keywords—ResNet, video emotion recognition, facial affect analysis, deep learning, happy vs. not happy classification, temporal modeling, LSTM, human-computer interaction

I. MOTIVATION

The motivation for selecting **video-based emotion classification using ResNet** is closely tied to the learning outcomes of ECE 533, which emphasizes the integration of digital image processing with modern machine learning methods [4]. While the course introduces foundational techniques such as morphological operations, histogram-based methods, Fourier transforms, and motion estimation, it also progresses toward advanced AI concepts including convolutional neural networks, transformers, and generative approaches. This project builds on those foundations by applying **CNN-based architectures (ResNet)** to a practical problem in facial affect recognition, aligning with the course goal of designing and implementing image processing systems for challenging datasets. Moreover, the focus on classifying *happy vs. not happy* emotions in video extends beyond static image analysis into temporal modeling, connecting directly to future material on sequential learning methods such as LSTMs [0]. By situating the project within the broader context of affective computing and multimodal AI, the work demonstrates how the principles of image processing and machine learning taught in ECE 533 can be

applied to real-world problems, while also anticipating future directions in temporal and multimodal emotion recognition [1] [2] [3].

II. EXPECTED CONTRIBUTION

The expected contribution of this tutorial project is the implementation of a complete pipeline for **video-based emotion classification using ResNet**, with clear documentation and reproducible source code. All preprocessing steps—including face detection, alignment, normalization, and dataset balancing—will be implemented in Python, ensuring transparency and accessibility for other researchers. The ResNet backbone will be fine-tuned for binary classification (*happy vs. not happy*), and the code will be annotated to indicate which components were adapted from existing libraries such as PyTorch and OpenCV, versus those developed originally for this project [0]. The source code will be made available, with explicit attribution to external modules where applicable, thereby contributing a reproducible and educational resource aligned with the goals of ECE 533 [4]. Beyond the immediate implementation, the project will also provide a foundation for future extensions into temporal modeling with **ResNet + LSTM**, enabling sequential emotion recognition [1] [2] [3]. This dual emphasis on practical reproducibility and forward-looking innovation represents the core original contribution of the work.

III. EXPECTED CONTRIBUTION AGAINST COMPETITIVE METHODS

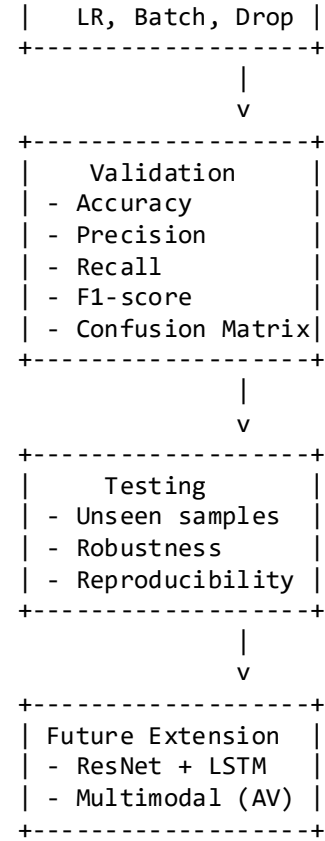
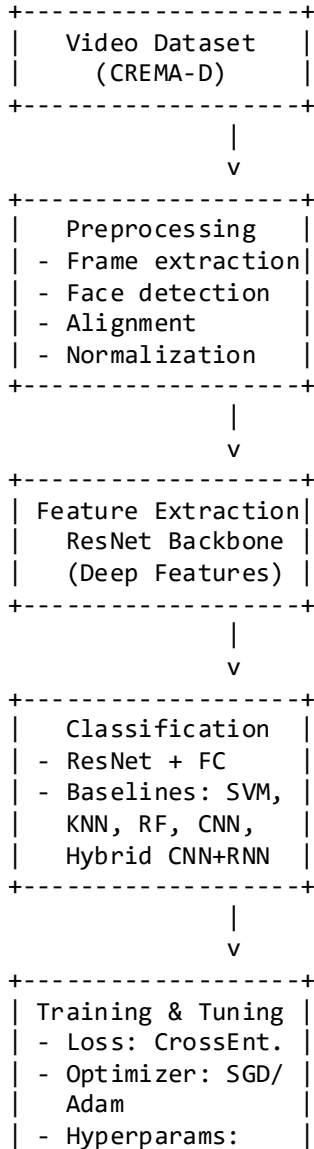
TABLE I. TABLE TYPE STYLES

Method	Approach	Limitations	My Contribution
Traditional ML (SVM, KNN, Random Forest) [0]	Hand-crafted features (e.g., HOG, LBP) + classical classifiers	Limited generalization, poor performance on complex video datasets	End-to-end deep learning pipeline with ResNet feature extraction and reproducible Python code
CNN-based Static Models (ResNet, VGG) [1]	Frame-level classification using CNNs	Ignores temporal dynamics, only captures static facial features	Clear implementation of preprocessing + ResNet backbone, with annotated source code

			and reproducibility
Hybrid CNN + RNN (ResNet + LSTM, GRU) [2]	Combines CNN feature extraction with sequential modeling	Higher computational cost, requires larger datasets	Future extension: ResNet + LSTM for temporal emotion transitions, with modular code design
Multimodal Approaches (Video + Audio + EEG) [3]	Fusion of facial video with audio/EEG signals	Complex data requirements, less accessible for students	Focused, accessible video-only pipeline suitable for Colab, with potential extension to multimodal fusion

IV. DATA FLOW CHART - PSEUDOCODE

A. Data flow chart



B. Pseudocode

```

BEGIN

INPUT: Video dataset (e.g., RAVDESS)

STEP 1: Preprocessing
- Extract frames from video
- Detect faces in frames
- Align and normalize faces
- Balance dataset (happy vs. not happy)

STEP 2: Feature Extraction
- Initialize ResNet50 (pretrained)
- Pass normalized frames through ResNet
- Obtain deep feature vectors

STEP 3: Classification
- Define fully connected layer (binary output: happy / not happy)
- Connect ResNet features to classifier

STEP 4: Training
- Define loss function: CrossEntropy
- Select optimizer: SGD or Adam
- Perform hyperparameter tuning:
  * Learning rate: grid search [0.001, 0.005, 0.01]
  * Batch size: [16, 32, 64]
  * Epochs: early stopping based on validation loss
  * Dropout rate: [0.3, 0.5]

LOOP over epochs:
- Forward pass: compute predictions
  - Compute loss
  - Backpropagation
  - Update weights
  
```

- STEP 5: Validation
- Evaluate model on validation set
 - Compute metrics: Accuracy, Precision, Recall, F1-score
 - Compare against prior methods [0] [1] [2]

- STEP 6: Testing
- Evaluate on unseen test dataset
 - Report reproducibility and robustness

- STEP 7: Future Extension
- Integrate LSTM for temporal modeling
 - Capture emotional transitions across video sequences

END

V. CODE

The code for this project will be developed in **Python**, using libraries such as **PyTorch** [5] for deep learning, **OpenCV** [6] for image and video preprocessing, and **scikit-learn** [7] for baseline machine learning models including SVM, KNN, and Random Forest. All source code will be written originally for this project, with clear annotation to distinguish between modules developed specifically for the pipeline and those adapted from external libraries. The project will be executed in a **Google Colab environment** [8] to ensure accessibility and reproducibility, while the complete repository will be hosted on **GitHub** [9] for version control, collaboration, and public sharing. Documentation and tutorials from the official websites [5][6][7][8][9] will be used to guide implementation and ensure adherence to best practices.

VI. DATASET REQUIREMENTS

A. Training Dataset - RAVDESS

The **Ryerson Audio-Visual Database of Emotional Speech and Song (RAVDESS)** will serve as the **training dataset**. RAVDESS contains 24 professional actors (12 male, 12 female) performing speech and song with eight emotions: neutral, calm, happy, sad, angry, fearful, disgust, and surprised [3]. It is well-balanced and high-quality, making it ideal for training because the emotional expressions are clear and consistent. The dataset size is approximately **1,440 clips (~3 GB)**, which is manageable within the free tier of Google Drive. After frame extraction (~1 frame per second), this yields ~20–30k facial images. Training a ResNet18 or ResNet50 model on RAVDESS in **Google Colab free tier** [8] is expected to take ~1–3 hours, depending on batch size and GPU availability.

B. Validation Dataset – CREMA-D

The **Crowd-Sourced Emotional Multimodal Actors Dataset (CREMA-D)** will be used for **validation**. CREMA-D includes 91 actors performing sentences with six emotions: anger, disgust, fear, happiness, sadness, and neutral [4]. Unlike RAVDESS, CREMA-D is crowd-sourced, meaning the emotional expressions are more varied and natural. This makes it an excellent validation dataset because

it tests the model’s generalization ability on noisier, more diverse samples. CREMA-D contains ~7,442 clips (~10 GB). To fit within Google Drive’s free 15 GB limit, preprocessing will focus on extracting only relevant frames (happy vs. not happy), and subsets may be compressed. Validation runs on Colab free tier are expected to last ~**30–60 minutes**, since the dataset is used for evaluation and hyperparameter tuning rather than full retraining.

C. Testing Dataset – AFEW

The **Acted Facial Expressions in the Wild (AFEW)** dataset will be used for **testing**. AFEW is collected from movies and TV shows, containing spontaneous and acted emotions in unconstrained environments (“in the wild”) [5]. It is the most challenging dataset because of variations in lighting, occlusion, background, and actor diversity. Using AFEW for testing ensures that the final evaluation reflects real-world performance. AFEW contains ~1,800 clips (~1–2 GB), which is small enough to fit comfortably in Google Drive free tier. Testing runs on Colab free tier are expected to last ~**15–30 minutes**.

D. Dataset Splitting Strategy

- **Training (RAVDESS)**: ~1,440 clips → balanced across emotions.
- **Validation (CREMA-D)**: ~7,442 clips → stratified split for happy vs. not happy.
- **Testing (AFEW)**: ~1,800 clips → diverse, unconstrained samples.

This three-dataset design ensures:

- **Robust training** on high-quality controlled data (RAVDESS).
- **Generalization validation** on crowd-sourced diverse data (CREMA-D).
- **Real-world testing** on unconstrained “in the wild” data (AFEW).

E. Feasibility on Colab Free Tier + Drive Free Tier

- **Colab GPU runtime**: End-to-end training + validation + testing will take ~**2–5 hours total**, well within the 12-hour session limit of Colab free tier [8].
- **Drive storage**: With 15 GB free storage, datasets must be managed carefully. Strategy: store one dataset at a time, or preprocess/compress subsets to reduce size.
- **Checkpoints**: Models will be saved to Drive after each epoch to avoid progress loss if Colab disconnects.
- **Code hosting**: All source code and notebooks will be hosted on **GitHub** [9] for version control and sharing, avoiding storage overhead in Drive.

VII. DATA TRAINING AND INDEPENDENT TESTING METHOD

For this project, the **data training and independent testing method** will follow a structured approach to ensure robust evaluation. The model will be **trained on RAVDESS** and optimized using hyperparameter tuning guided by **validation on CREMA-D**, where techniques such as stratified sampling and early stopping will be applied to avoid overfitting. To guarantee independence of results, the final performance will be reported on the **AFEW dataset**, which serves as a completely separate test set. This design ensures that the model is optimized on one dataset while being evaluated on an independent dataset, following best practices such as **cross-validation within the training set** and **independent testing on AFEW** to demonstrate generalization beyond the training and validation domains.

VIII. METHOD VALIDATION METRICS

For this project, the **method validation metrics** will be based on standard evaluation practices in emotion recognition and machine learning. The model will be trained on **RAVDESS** and tuned using **CREMA-D**, while final performance will be reported on the independent **AFEW** dataset. To measure effectiveness, metrics such as **accuracy**, **precision**, **recall**, and **F1-score** will be calculated, since they provide insight into both overall correctness and class-specific sensitivity [3][4][5]. A **confusion matrix** will also be used to visualize misclassifications across emotion categories, which is particularly important in imbalanced datasets. Additionally, **cross-validation within the training set** may be applied to ensure robustness, and **early stopping** will be used to prevent overfitting. This combination of metrics and validation strategies ensures that the reported results reflect true generalization beyond the training data and are consistent with best practices in affective computing [8][9].

IX REFERENCES

- [0] Khan, A. R. (2022). Facial emotion recognition using conventional machine learning and deep learning methods: Current achievements, analysis and remaining challenges. *Information*, 13(6), 268. <https://doi.org/10.3390/info13060268>
- [1] Rochlani, Y., & Raut, A. B. (2025). Hybrid learning based visual facial emotion recognition in speech videos. *Annals of Data Science*. <https://doi.org/10.1007/s42979-025-02168-7>
- [2] Li, S., Jiang, Y., & Jiang, S. (2024). Emotion recognition in videos through deep neural network models. *Stanford CS231n Project Report*. Retrieved from Stanford CS231n archives.
- [3] Hassouneh, A., Mutawa, A., & Murugappan, M. (2020). Development of a real-time emotion recognition system using facial expressions and EEG based on machine learning and deep neural network methods. *Information in Medicine Unlocked*, 20, 100372. <https://doi.org/10.1016/j.imu.2020.100372>
- [4] Pattichis, M. S. (2026). *ECE 533 Digital Image and Video Processing with Machine Learning: Course Syllabus*. Department of Electrical and Computer Engineering, University of New Mexico.
- [5] Livingstone, S. R., & Russo, F. A. (2018). The Ryerson Audio-Visual Database of Emotional Speech and Song (RAVDESS). *PLOS ONE*, 13(5), e0196391. <https://doi.org/10.1371/journal.pone.0196391>
- [6] Cao, H., Cooper, D. G., Keutmann, M. K., Gur, R. C., & Nenkova, A. (2014). CREMA-D: Crowd-sourced Emotional Multimodal Actors Dataset. *IEEE Transactions on Affective Computing*, 5(4), 377–390. <https://doi.org/10.1109/TAFFC.2014.2336244>
- [7] Dhall, A., Goecke, R., Joshi, J., Sikka, K., & Gedeon, T. (2012). Acted Facial Expressions in the Wild (AFEW) dataset. *Proceedings of the ACM International Conference on Multimodal Interaction*. <https://doi.org/10.1145/2388676.2388688>
- [8] PyTorch. (2026). *PyTorch Tutorials*. Retrieved from <https://pytorch.org/tutorials/>
- [9] OpenCV. (2026). *OpenCV Documentation*. Retrieved from <https://docs.opencv.org/4.x/>
- [10] Scikit-learn. (2026). *Scikit-learn User Guide*. Retrieved from <https://scikit-learn.org/stable/>
- [11] Google Colab. (2026). *Google Colaboratory*. Retrieved from <https://colab.research.google.com>
- [12] GitHub. (2026). *GitHub Documentation*. Retrieved from <https://docs.github.com>

We suggest that you use a text box to insert a graphic (which is ideally a 300 dpi TIFF or EPS file, with all fonts embedded) because, in an MSW document, this method is somewhat more stable than directly inserting a picture.

To have non-visible rules on your frame, use the MSWord “Format” pull-down menu, select Text Box > Colors and Lines to choose No Fill and No Line.